Module 09 Lab - Ensemble Methods**Objective:** To learn how to use **ensemble methods**, which combine the predictions of several individual models to produce a final prediction that is more accurate and robust than any of the individual models alone.**In this lab, you will train and compare a single Decision Tree against a powerful ensemble method, the Random Forest.**

Part 1: The Wisdom of the Crowd**Concept:** The core idea behind ensemble methods is the "wisdom of the crowd." If you ask one person a difficult question, they might be wrong. But if you ask a large, diverse group of people the same question and average their answers, the result is often surprisingly accurate.Ensemble methods apply this to machine learning. Instead of relying on a single model, we build a "committee" of models and have them vote on the final prediction.Two popular ensemble techniques:* **Bagging (Bootstrap Aggregating):** Trains multiple models (e.g., Decision Trees) on different random subsets of the training data. The **Random Forest** is the most famous example.* **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by the previous one. Examples include Gradient Boosting and

AdaBoost.

Part 2: SetupWe will use the Iris dataset to classify flower species.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare data
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0
print(f"Training samples: {len(X_train)}; Test samples: {len(X_test)}"
```

```
Training samples: 105; Test samples: 45
```

Part 3: A Single Decision TreeA single Decision Tree is prone to overfitting. It can create very specific rules that match the training data perfectly but don't generalize well.**Your Task:** Train a single `DecisionTreeClassifier` and evaluate its accuracy.

```
# Train and evaluate one Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)
y_pred_tree = single_tree.predict(X_test)
accuracy_tree = accuracy_score(y_test, y_pred_tree)
print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")
```

```
Accuracy of a single Decision Tree: 100.00%
```

Part 4: The Random Forest Ensemble**Concept:** A Random Forest is an ensemble of many Decision Trees. It builds

hundreds of trees, each trained on a random subset of the data and using a random subset of the features. To make a prediction, it gets a vote from every tree in the "forest" and chooses the class that gets the most votes.This process of averaging the votes of many diverse trees makes the Random Forest much more robust and less prone to overfitting than a single tree.**Your Task:** Train a `RandomForestClassifier` and compare its accuracy to the single tree.

```
# Train and evaluate a Random Forest with 100 trees
random_forest = RandomForestClassifier(n_estimators=100, random_state=
random_forest.fit(X_train, y_train)
y_pred_rf = random_forest.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")
```

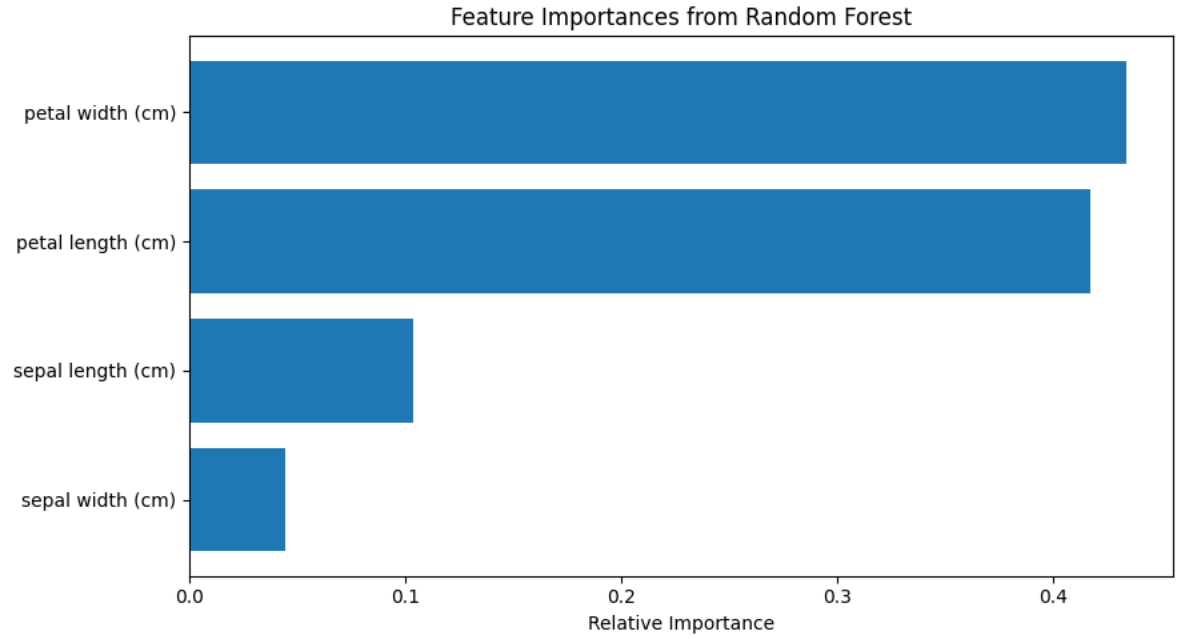
```
Accuracy of the Random Forest: 100.00%
```

Part 5: Feature Importance**Concept:** A great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions.**Your Task:** Extract and plot the feature importances from your trained Random Forest model.

```
import matplotlib.pyplot as plt
import numpy as np

# Show which flower measurements were most useful to the forest
importances = random_forest.feature_importances_
feature_names = iris.feature_names
indices = np.argsort(importances)
plt.figure(figsize=(9, 5))
plt.title("Feature Importances from Random Forest")
```

```
plt.barh(range(len(indices)), importances[indices], align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.tight_layout()
plt.show()
for i in indices[::-1]:
    print(f"{feature_names[i]}: {importances[i]:.3f}")
```




```
petal width (cm): 0.434
petal length (cm): 0.417
sepal length (cm): 0.104
sepal width (cm): 0.045
```

Knowledge Check

1. What is the main idea behind ensemble methods?

Ensemble methods combine predictions from multiple models instead of relying on one. A group of trees can make a more dependable prediction because individual trees may make different mistakes.

2. Which model performed better?

The Decision Tree scored 100.00% and the Random Forest scored 100.00% on the

The Decision Tree scored 100.00% and the Random Forest scored 100.00% on the test set.

3. Which two features were most important?

The two highest were petal width (cm) and petal length (cm).